

Understanding Semantic IDs: From Item Representation to Item Selection in Generative Recommendation

Junting Wang
junting3@illinois.edu

University of Illinois Urbana-Champaign
Urbana, Illinois, USA

Yunzhe Li
yunzhel2@illinois.edu

University of Illinois Urbana-Champaign
Urbana, Illinois, USA

Xinrui He

xhe33@illinois.edu

University of Illinois Urbana-Champaign
Urbana, Illinois, USA

Hari Sundaram

hs1@illinois.edu

University of Illinois Urbana-Champaign
Urbana, Illinois, USA

ABSTRACT

Semantic IDs (SIDs) are now a central component of generative recommendation. Current SID-based systems assign three roles to the same token sequence. Shared prefixes are intended to organize related items, the complete SID identifies an individual item, and each generated token narrows the items that can still be returned. We systematically investigate SIDs from item encoding and SID construction to autoregressive generation and final recommendation. We examine how SID construction changes item representations and how those changes affect generation. Across three Amazon domains and eight SID constructions, SID neighborhoods recover only 32.2% of the encoder’s ten nearest neighbors on average. Alternative item descriptions still retrieve the corresponding item first in 99.57% of controlled cases, yet change 38.4% of exact SIDs. These results show that SIDs retain broad organization but lose much of the encoder’s fine local structure, while their exact tokens are not determined by item meaning alone. This loss becomes consequential during generation. After the final semantic token, TIGER retains only 29.9% of held-out targets that were plausible recommendations before SID filtering. Motivated by these findings, we propose Item-Supported Decoding (ISD), a lightweight inference-time method that allows a user-specific item ranking to support corresponding SID prefixes before beam search discards them. The same ranking then orders the generated items. ISD requires no additional parameters or retraining of the SID constructor or decoder. We empirically show that ISD improves NDCG@10 over the corresponding SID backbone in every evaluated setting, with relative gains of up to 31.2%. Our results show that SIDs provide useful coarse item organization, but their fine boundaries should not alone determine which items remain available during generation. Code and experimental artifacts are available at <https://anonymous.4open.science/r/ISD-CF7C>.

KEYWORDS

Generative Recommendation, Semantic IDs, Discrete Item Representations, Quantization

1 INTRODUCTION

Generative recommenders increasingly use Semantic IDs (SIDs) to represent items [2, 5, 12, 15, 21]. They use codebooks to convert item content into short sequences of discrete tokens. The motivation is compelling: content guides how items are tokenized, while

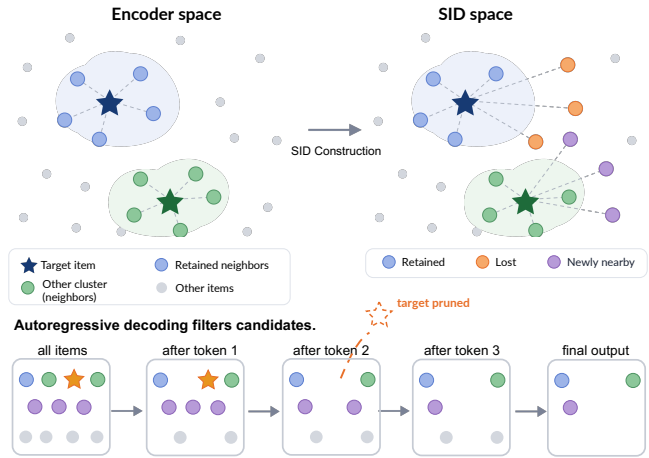


Figure 1: SIDs preserve broad semantic organization but lose fine-grained local item structure.

the shared codebook allows related products to share tokens or prefixes. Therefore, during training, the interaction data can reinforce common parts of the prediction instead of teaching the model to recognize every item as an unrelated label.

Existing SID recommenders use shared tokens to encode common semantic or collaborative structure [15, 21], while treating the complete token sequence as the item identifier to be generated [2, 12, 15]. This places two demands on the same representation. The shared tokens should organize related items so that they can benefit from common structure, while the complete sequence must uniquely identify the item. During decoding, autoregressive SID recommenders generate the prediction sequence using constrained beam search [12, 15, 21]. Autoregressive generation therefore gives the SID a third role in item selection. At each decoding step, the predicted token retains only the items with that prefix and removes the rest. Therefore, the distinctions introduced to identify individual items also become intermediate decisions about which items remain eligible for recommendation.

We ask two questions. *What changes when item representations are converted into SIDs? How do the resulting SIDs shape item selection during generation?* We answer the first by testing whether equivalent descriptions produce the same exact SID and whether

the organization of the continuous item space survives discretization. We answer the second by tracing how the set of items that can still be recommended changes after each generated token.

To study how item representations change during SID construction, we examine both the content encoder and the SID constructor (§ 3). Following the principle of controlled behavioral testing [17], for each item, we create an alternative description containing exactly the same words but presenting its fields in a different order, such as placing the title before rather than after the description. Across three encoder families, the original and alternative descriptions have an average cosine similarity of approximately 0.99. In nearest-neighbor retrieval over all original item representations, the alternative representation retrieves its corresponding item first in 99.57% of the Office and Scientific cases. Across eight SID constructions and three domains, 38.4% of items receive a different exact SID on average, while the resulting code spaces recover only 32.2% of the encoder’s ten nearest neighbors. Thus, SIDs preserve broad organization but lose much of the encoder’s fine structure, and semantic identity alone does not determine their exact tokens.

To study how the resulting SIDs shape item selection, we follow each held-out target through SID generation and record the token at which it can no longer be returned (§ 4). Each generated token narrows the remaining item set, and removing an SID prefix permanently removes every item whose SID begins with it. Only 5.1% of targets remain before final ranking. A separate recommender then ranks all eligible items. With the SIDs and beam size fixed, allowing highly ranked items to support their prefixes raises target availability to 8.5%.

Together, these results expose a mismatch between how SIDs organize items and how they are used during generation. SID tokens must distinguish individual items, yet beam search also uses those tokens to decide which items remain eligible. A distinction needed for identification can therefore become a premature relevance decision. This motivates our method, using SIDs to organize and identify items while introducing separate item-level evidence to determine which items remain under consideration.

Present work. We propose *Item-Supported Decoding* (ISD), a lightweight decoding intervention that requires no additional parameters or retraining. Given an item ranking derived from the user’s history, ISD gives additional weight to SID prefixes associated with highly ranked items before beam reduction. A prefix can therefore remain because the decoder favors it or because it contains an item ranked highly for the user. After generation, ISD uses the same ranking to order the generated items.

Recent work reduces dependence on early SID decisions through architectural changes. SETRec predicts an order-agnostic token set instead of an ordered SID [11]. LIGER retains SID generation but adds dense retrieval as a second path for missed items [23]. ISD instead operates inside an existing autoregressive decoder before beam reduction makes a token decision irreversible. Across the evaluated systems, ISD improves NDCG@10 by up to 31.2%. Our **key contributions** are as follows:

Controlled Investigation of SID Structure: We introduce a same-item evaluation that separates continuous item recognition, exact SID assignment, and local-structure preservation. Unlike evaluations based only on reconstruction or recommendation accuracy, our analysis tests whether descriptions containing the

same item facts remain equivalent after SID construction. Across three domains and eight SID constructions, we show that current SIDs preserve broad semantic organization but lose much of the encoder’s fine local structure, while their exact tokens are not determined by semantic identity alone.

Consequences for Autoregressive Generation: We connect SID structure to generation by tracing held-out targets after every semantic token. Our analysis shows how fine SID distinctions become irreversible item-selection decisions during beam search. A controlled intervention further demonstrates that allowing item-level evidence to act before beam reduction keeps more ground-truth targets available under the same SIDs and beam size.

Item-Supported Decoding: Based on these findings, we propose ISD, a lightweight inference-time method that requires no retraining of the SID constructor or decoder. ISD accepts an item ranking from any ranking method, uses it to support relevant SID prefixes before beam reduction, and orders only the generated items afterward. Across TIGER and LIGER, three domains, and rankings produced by training statistics, SASRec, and UniSRec, ISD consistently improves NDCG@10, with gains of up to 31.2%.

2 PRELIMINARIES

This section defines the generative recommendation task and the notation used throughout the paper.

Generative recommendation. Let $\mathcal{I} = \{1, \dots, N\}$ be the item set and $\mathcal{U}$ the user set. For a user $u \in \mathcal{U}$, the interaction history $H_u = (i_1, \dots, i_T)$ is an ordered sequence of items from $\mathcal{I}$. Given H_u , the next-item recommendation task is to rank a target item $i^+ \in \mathcal{I}$ ahead of the remaining items. We write $\mathcal{D} = \{(H_u, i^+)\}$ for the training examples.

Semantic IDs. Each item i is associated with content x_i , such as its title, brand, category, or description. An SID pipeline encodes this content as a continuous representation $h_i = f(x_i)$ and discretizes it into a semantic tuple $z_i = q(h_i) = (z_{i,1}, \dots, z_{i,L})$ [10, 15, 21]. Some constructors additionally apply a collision-resolution or uniqueness rule a [15]; we therefore write

$$h_i = f(x_i), \quad z_i = q(h_i), \quad C_i = a(z_i, i), \quad (1)$$

where $C_i = (c_{i,1}, \dots, c_{i,L_i})$ is the complete SID used by the recommender. When no additional rule is required, a is the identity map and $C_i = z_i$. This notation separates the continuous item representation h_i , the content-derived discrete tuple z_i , and the complete item sequence C_i presented to the recommendation model.

SID-based prediction. Let $C(H_u) = (C_{i_1}, \dots, C_{i_T})$ be the SID representation of a user’s history. A generative recommender assigns probability $p_\theta(C_i | C(H_u))$ to an item’s complete SID and is trained by [15]

$$\mathcal{L}_{\text{rec}}(\theta) = - \sum_{(H_u, i^+) \in \mathcal{D}} \log p_\theta(C_{i^+} | C(H_u)). \quad (2)$$

For an autoregressive decoder, this sequence probability factorizes as $p_\theta(C_i | C(H_u)) = \prod_{\ell=1}^{L_i} p_\theta(c_{i,\ell} | C(H_u), c_{i,<\ell})$ [15]; parallel decoders use a different factorization but predict the same complete item sequence [5]. At inference, generated SIDs are resolved to items through the system’s SID-to-item map, and items are ranked

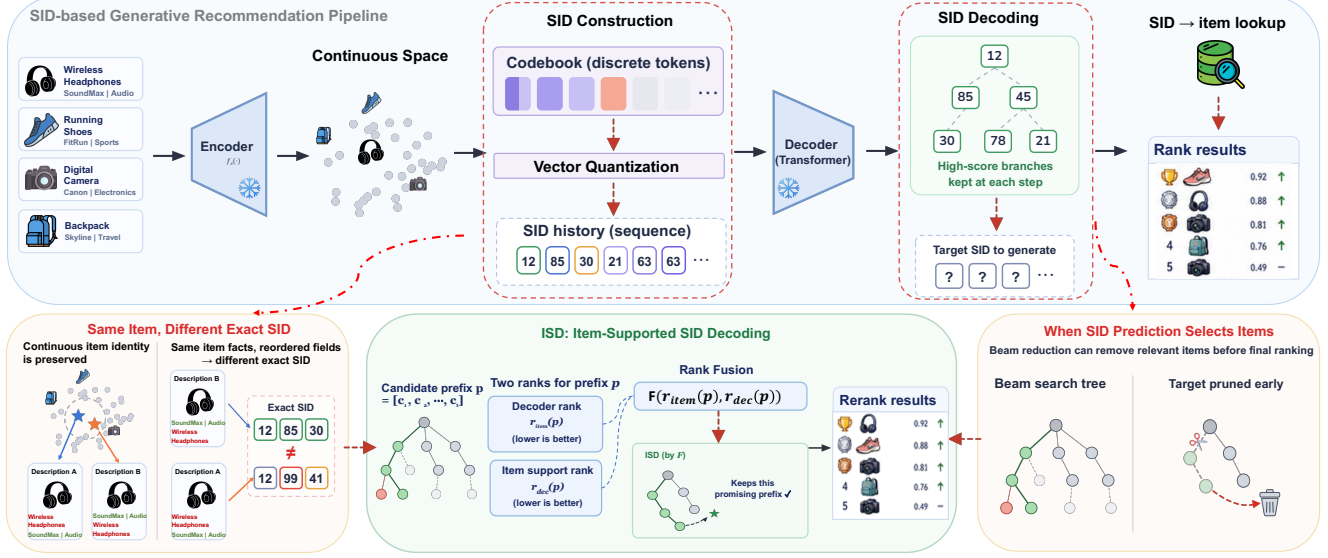


Figure 2: Overview of SID construction, item filtering, and Item-Supported Decoding (ISD).

by their sequence scores. Later sections distinguish the semantic tuple from the complete SID whenever a constructor or decoding rule makes that distinction necessary.

3 WHAT SEMANTIC IDS PRESERVE AND LOSE

Semantic IDs rest on a common premise: item content can provide both the structure used to share information across related items and the discrete label used to identify an item during generation [2, 5, 12, 15, 21]. SIDs are expected to put similar items under similar codes while assigning an identifiable sequence to each item. We study what changes during this conversion from two perspectives. First, we test whether equivalent descriptions of the same item produce the same exact SID. Second, we test how much of the continuous item-space organization remains after SID construction. These properties matter because the exact SID is the supervised prediction target, while shared structure is intended to let related items share statistical strength.

We trace items through content encoding (§ 3.1), SID construction, and decoding. The constructor comparison is reported in Table 2, while Table 1 provides the continuous encoder controls.

3.1 Exact SID Assignment

This section tests what determines an item’s exact SID. We test what happens when the same item facts are presented in different orders. If an SID depends only on the item’s semantic meaning, its assigned code should remain unchanged.

3.1.1 Analysis Setup. Following behavioral invariance testing [17], we change presentation while preserving the item and its contents. Let $\mathcal{X}$ denote the set of item descriptions and let $\pi(x) \in \mathcal{I}$ be the item described by x . Starting from source description x_i , we construct $x'_i = \tau(x_i)$ by changing only the order in which the same item information, such as its title and description, is presented. No words or attribute values are added, removed, or changed. We write $x \sim x'$ when two descriptions refer to the same item, so that

Table 1: Encoder stability (%).

Text encoder	Office		Scientific	
	Identity@1	R_{Enc}^{10}	Identity@1	R_{Enc}^{10}
Sentence-T5	99.51	91.81	99.51	92.72
BGE-base	99.46	91.55	99.46	92.87
Qwen3-0.6B	98.97	86.10	99.41	89.96

$\pi(x) = \pi(x')$. Thus, every evaluated pair satisfies

$$x_i \sim x'_i, \quad \text{bag}(x_i) = \text{bag}(x'_i), \quad \pi(x_i) = \pi(x'_i) = i, \quad (3)$$

where $\text{bag}(x)$ is the multiset of tokens and field values in x . Thus x_i and x'_i differ only in presentation. Let $\mathcal{P}$ be the set of item indices represented by these matched-description pairs. If the complete SID $C(x)$ identifies the item directly, it must be constant over such matched descriptions:

$$x \sim x' \implies C(x) = C(x'). \quad (4)$$

3.1.2 Metric and Interpretation. For each $i \in \mathcal{P}$, let $C(x_i)$ and $C(x'_i)$ be the complete SIDs produced by the same frozen constructor. We measure

$$M_i = 1[C(x_i) \neq C(x'_i)], \quad \Delta_{\text{SID}} = \frac{1}{|\mathcal{P}|} \sum_{i \in \mathcal{P}} M_i. \quad (5)$$

Intuitively, $M_i = 1$ means that this order change gives item i a different complete SID, i.e., the token sequence that the recommender is trained to generate. Δ_{SID} is the proportion of matched items for which this occurs. We present the results in Table 2. The RQ-VAE-latent control applies RQ-KMeans to the representation learned by RQ-VAE, changing the quantizer while holding its input fixed.

3.1.3 Encoder-Space Control. Before measuring SID changes, we verify that reordering preserves the item’s identity in the continuous embedding space. Let $e_i = f(x_i)$ and $e'_i = f(x'_i)$ denote the

Table 2: Exact-code change and top-10 neighborhood preservation, using Sentence-T5 as the encoder

SID constructor	Baby (%)			Office (%)			Scientific (%)		
	$\Delta\text{SID} \downarrow$	Enc-SID $F^{10} \uparrow$	SID-SID' $R_{\text{SID}}^{10} \uparrow$	$\Delta\text{SID} \downarrow$	Enc-SID $F^{10} \uparrow$	SID-SID' $R_{\text{SID}}^{10} \uparrow$	$\Delta\text{SID} \downarrow$	Enc-SID $F^{10} \uparrow$	SID-SID' $R_{\text{SID}}^{10} \uparrow$
RQ-VAE [10, 15]	54.5	19.9	71.1	53.9	13.8	63.2	45.7	32.8	80.4
RQ-KMeans [14]	20.4	36.5	94.6	29.3	31.5	87.0	17.5	43.2	96.5
RQ-KMeans (normalized) [14]	17.4	38.9	96.0	24.6	33.1	89.0	14.0	45.1	97.6
RQ-KMeans (RQ-VAE latent) [10, 14]	50.8	23.8	79.5	63.8	18.7	66.9	41.2	37.5	87.5
DRQ [22]	35.4	33.9	85.2	42.3	27.9	76.4	29.5	44.3	90.8
RQP-VAE [22]	73.5	36.6	79.0	58.0	32.1	73.2	67.0	45.0	84.2
OPQ/PQ [3, 5]	26.3	26.1	86.7	20.7	27.7	88.4	29.4	18.3	80.7
Balanced assignment [2]	33.2	36.6	87.7	36.2	28.2	82.5	38.1	40.6	88.9

representations of item i under its original and reordered descriptions. We measure their cosine similarity and use e'_i to query the original item representations $\{e_j = f(x_j)\}_{j \in \mathcal{I}}$. We define identity-retrieval accuracy as

$$\text{Identity@1} = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \mathbb{I} \left[i = \arg \max_{j \in \mathcal{I}} \cos(e'_i, e_j) \right]. \quad (6)$$

The reordered representation e'_i is not included in the item index. Under Sentence-T5, the mean cosine similarity between e_i and e'_i is 0.9939. Across Sentence-T5, BGE, and Qwen, the reordered description retrieves the original item first in 98.97–99.51% of fixed-anchor cases and retains 86.10–92.87% of its original top-10 neighbors. Thus, the encoders continue to recognize the same item and largely preserve its local neighborhood. Table 1 reports these controls.

3.1.4 Exact SIDs Are Not Determined by Meaning Alone. Reordering the same item facts preserves continuous item recognition, *i.e.*, the reordered description retrieves the original item first in 99.57% of controlled cases. However, 14.0–73.5% of items receive a different SID across constructors and domains (Table 2). Thus, recognizing the same item does not guarantee the same discrete code. Since this code is used as the generation target, the decoder must learn the constructor’s chosen assignment in addition to the item relationships encoded by the SID.

3.2 Local-Structure Preservation

In this section, we measure how faithfully SIDs preserve the local structure of the continuous item space from which they are built.

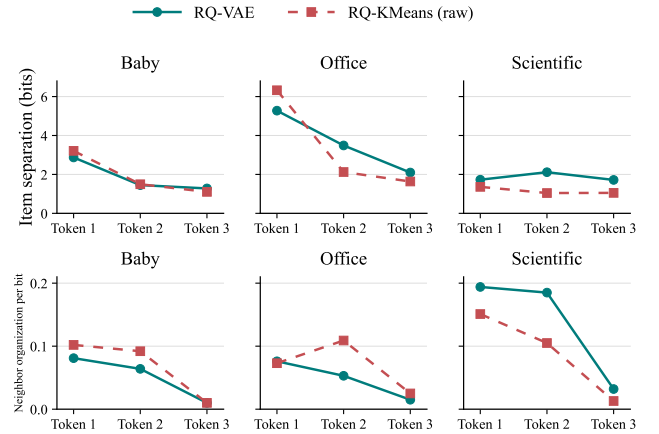
3.2.1 Metrics and Interpretation. For item i , let $N_E^k(x_i)$ be its self-excluded top- k encoder neighbors and $N_S^k(x_i)$ its top- k neighbors after SID reconstruction. We define

$$F_i^k = \frac{|N_E^k(x_i) \cap N_S^k(x_i)|}{k}, \quad R_{\text{SID},i}^k = \frac{|N_S^k(x_i) \cap N_S^k(x'_i)|}{k}. \quad (7)$$

We report their averages over the evaluated items as F^k and R_{SID}^k . F_i^k measures encoder-neighbor recovery for item i , while $R_{\text{SID},i}^k$ measures whether its SID neighbors survive the field reorder. The corresponding encoder-only control, $R_{\text{Enc}}^k = |N_E^k(x_i) \cap N_E^k(x'_i)|/k$, appears in Table 1.

3.2.2 SIDs Preserve Broad Organization, Not Fine Neighborhoods. Table 2 shows that even before the order test, SID reconstructions recover only 13.8–45.1% of the encoder’s exact top-10 neighbors. Additional ranking and distance checks show

that missing exact neighbors are usually replaced by other nearby items (§ A.1.2). The SIDs therefore retain broad organization while substantially rewriting the fine local structure from which they were built. The same distinction appears under the order test: SID neighborhoods retain 63.2–97.6% of their members, compared with 91.8–92.7% in the continuous-space control. § A.1.3 provides the corresponding geometry visualizations.

**Figure 3: Item separation and source-neighbor organization across the three semantic tokens.**

3.3 Coarse Organization and Fine Identification

The analyses in § 3.1 and 3.2 show that complete SIDs preserve broad organization but rewrite many fine item relationships. We next measure how item separation and neighbor preservation change across the SID. Following [2, 15], we set the token length to three and measure at each semantic position how much the next token reduces the possible items and whether encoder-space neighbors remain together. We exclude the collision-resolution suffix. § A.1.4 defines the measurements and randomization protocol.

Across both constructors and all three datasets, the first two tokens reduce the possible items while keeping substantially more encoder-space neighbors together. The final token continues to reduce the possible items, but keeps few additional neighbors together beyond the randomized expectation (Figure 3). It therefore contributes mainly to distinguishing items within groups formed by the earlier tokens.

Separating items is necessary to identify the final recommendation. The concern is that autoregressive decoding applies this separation before the complete SID is known. At each position, beam search discards some SID prefixes and removes every item whose identifier begins with a discarded prefix. § 4 measures how many otherwise plausible targets are lost during this process.

4 WHEN SID PREDICTION SELECTS ITEMS

§ 3 shows that early SID tokens place many encoder-space neighbors under the same prefix, while later tokens increasingly separate individual items. Autoregressive generation treats every token as a selection step. After predicting each token, beam search retains a fixed number of SID prefixes and discards the rest. Discarding a prefix removes every item whose SID begins with it, even though the final item has not yet been selected. A low score on a later token can therefore remove the correct item before its full SID is generated or final ranking begins.

The SID constructor determines which items share each prefix. When two encoder-space neighbors receive different later tokens, the decoder must assign probability to two different token sequences rather than one shared prefix. Although the decoder receives the user’s history, beam search ranks each SID prefix as a whole rather than ranking the individual items assigned to it. If the target’s prefix is discarded, the target is removed even when a separate model would rank that item highly from the same history. § 4.2 measures how often this occurs.

Prior work has addressed early removal by changing the recommendation architecture. SETRec predicts an unordered token set instead of an ordered SID [11]. LIGER combines SID generation with dense retrieval and ranking [23]. We instead keep the SID assignment, decoder, and beam search fixed. We follow each held-out target through SID generation and record when its prefix leaves the beam. Section 4.2 identifies removed targets that a separate item model ranks among its top k items. Section 4.3 then tests whether using these item scores during beam selection keeps more targets available.

4.1 How SID Prediction Filters Items

Suppose the decoder has generated SID prefix p , the sequence of SID tokens produced so far. Let S_p be the set of items whose assigned SIDs begin with p . For a possible next token v , the smaller set $S_{p,v} \subseteq S_p$ contains the items whose SIDs begin with (p, v) . We write $\mathcal{V}(p) = \{v : S_{p,v} \neq \emptyset\}$ for the possible next tokens. The decoder scores each $v \in \mathcal{V}(p)$ using the user’s interaction history h and the complete prefix,

$$p_\theta(v \mid h, p). \quad (8)$$

Beam search expands each retained SID prefix with its possible next tokens. Its *beam width* B is the number of extended prefixes retained after each token [11, 15]. If $\mathcal{B}_\ell$ is the set of these B prefixes after semantic position ℓ , then the items still available for recommendation are

$$\mathcal{R}_\ell = \bigcup_{p \in \mathcal{B}_\ell} S_p. \quad (9)$$

Removing an SID prefix removes every item in its set S_p . If the held-out target is not in $\mathcal{R}_\ell$, no later token or final ranking step can

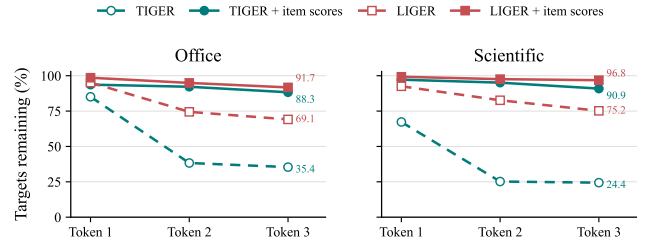


Figure 4: Top- k targets remaining after each semantic token, with $k = 10$. Adding the direct item score uses $\alpha = 0.75$. SID assignments and beam width are unchanged.

return it. The constructor therefore determines which items share each SID prefix. Beam search then uses those prefixes to decide which items remain available.

4.2 Are Highly Ranked Targets Removed Early?

SID generation removes some held-out targets before final item ranking begins. A removal affects ranking at cutoff k only when the target could otherwise appear among the top k items. We use a separate model that ranks items directly from the user’s history to identify these targets, then measure how many remain after each SID token.

4.2.1 Analysis Setup. We train a separate sequential item scorer $g(h, i)$ on training interactions. Given user history h , it ranks every unconsumed item i before SID generation removes any items. We retain cases in which this scorer places the held-out target i^* among its top k items. We then ask whether SID generation removes these highly ranked targets before final ranking. In the reported experiment, we set $k = 10$ to match Recall@10 and NDCG@10.

4.2.2 Metrics and Interpretation. Let $\mathcal{H}_k$ contain these top- k cases, and let $i^*(h)$ be the held-out target for history h . At each semantic position ℓ , we measure the fraction that remains available:

$$K_\ell = \frac{1}{|\mathcal{H}_k|} \sum_{h \in \mathcal{H}_k} \mathbf{1}[i^*(h) \in \mathcal{R}_\ell]. \quad (10)$$

Thus, K_ℓ is the fraction of these top- k targets that can still be returned after token ℓ . Appendix A.2 reports an additional analysis that gives more weight to targets ranked closer to first.

4.2.3 Removal Precedes Final Ranking. Figure 4 shows that both decoders remove top- k targets during SID generation, although TIGER removes more. With $k = 10$, after the third token TIGER retains 35.4% of the Office targets and 24.4% of the Scientific targets. LIGER retains 69.1% and 75.2%, respectively. Thus, SID prediction can remove targets that the separate scorer would place near the top before final item ranking begins.

4.3 Item Evidence Before Filtering

Figure 4 shows that SID generation removes many targets that the separate item scorer ranks highly. We next test whether these removals can be reduced without changing the SID assignments or retraining the decoder. The intervention increases the score of

an extended prefix when it contains an item ranked highly by the separate scorer.

4.3.1 Intervention Before Prefix Removal. At a decoding step, prefix p can be extended with any token v for which $S_{p,v}$ is nonempty. Ordinary beam search scores each extension (p, v) using the decoder probability. We additionally take the highest score among the unconsumed items that the extension would keep:

$$q(h, p, v) = \max_{i \in S_{p,v} \setminus h} g(h, i). \quad (11)$$

A large $q(h, p, v)$ means that choosing token v keeps at least one item that the separate scorer ranks highly. The held-out target identity is never supplied. Let $d_v = \log p_\theta(v | h, p)$ be the decoder score. We combine the two scores as

$$s_\alpha(h, p, v) = (1 - \alpha)\tilde{d}_v + \alpha\tilde{q}_v, \quad (12)$$

where $\tilde{d}_v$ and $\tilde{q}_v$ denote the normalized decoder and item scores across the possible next tokens.

4.3.2 Item Scores Preserve More Targets. Figure 4 shows that using a separate item scorer to guide beam search substantially increases the number of targets retained. However, the intervention is expensive because it scores every item at every token position. This motivates the efficient repair in § 5, which combines the decoder score with one additional item ranking.

5 ITEM-SUPPORTED DECODING

Sections 3 and 4 expose a conflict between the roles assigned to an SID. Shared prefixes provide a compact organization of the item set, but the fine tokens do not preserve every fine-grained item relationship. During generation, those same tokens become filtering decisions: removing an SID prefix removes all items assigned beneath it. Section 4 shows that many targets remain available when token selection also considers whether each token would preserve an item that a separate sequential recommender ranks highly from the user’s history. That diagnostic repeatedly scores every unconsumed item at every token position, making it too expensive to use as a recommender.

We turn that diagnostic into a lightweight repair called *Item-Supported Decoding* (ISD). Given the observed history, ISD constructs one ranked item set from a separate item ranker (§ 5.1). During SID generation, this ranking supports prefixes before beam reduction removes their items (§ 5.2.1). After complete SIDs are generated, the same ranking orders the generated items (§ 5.2.2).

5.1 Constructing the Ranked Item Set

Given the observed history h , an item ranker assigns a score $s(h, i)$ to every unconsumed item i . We sort the items by this score and retain the top J as the ranked item set $Q(h)$. The value J controls how many items can provide support during SID generation. We construct $Q(h)$ once before generation and use item ranks rather than raw scores. ISD can therefore accept rankings from models with different score scales without model-specific calibration [1]. We study two sources of item rankings. The first uses statistics computed from training interactions, while the second uses learned sequential recommenders.

5.1.1 Training-Statistics-Based Ranking. Our simplest ranking combines first-order item transitions and cosine-normalized item co-consumption computed from the training sequences [16, 18]. Let N_i be the number of occurrences of item i , let $T_{j,i}$ count how often i immediately follows j , and let $W_{x,i}$ count how often x and i occur within w positions of one another. We define

$$A_{j,i} = \log(1 + T_{j,i}), \quad C_{x,i} = \frac{W_{x,i}}{\sqrt{N_x N_i}}. \quad (13)$$

$A_{j,i}$ measures direct transitions. The logarithm prevents a few frequent transitions from dominating the score. $C_{x,i}$ measures nearby consumption, normalized so that two individually popular items do not receive a high score merely because both appear often. For history h , let j be its final observed item and let $H_m(h)$ contain its m most recent items. We score an unconsumed item i by

$$s_{\text{stat}}(h, i) = \frac{1}{2} z_h(A_{j,i}) + \frac{1}{2} z_h\left(\frac{1}{|H_m(h)|} \sum_{x \in H_m(h)} C_{x,i}\right), \quad (14)$$

Here, z_h standardizes each signal across the unconsumed items for history h . Appendix A.3 gives the exact rule. Ranking items by $s_{\text{stat}}(h, i)$ produces $Q(h)$. The statistics and scoring rule are fixed using training data. At evaluation, the score uses the observed input history but never the held-out next item.

5.1.2 Sequential-Model-Based Ranking. A trained sequential recommender provides the same interface. SASRec [8] scores items from learned item IDs, while UniSRec [6] also incorporates item content. For either model, $s(h, i)$ is its score for unconsumed item i . The three reported variants are therefore ISD (statistics), ISD (SASRec), and ISD (UniSRec). The SID model and fusion rule remain the same across ranking sources.

5.2 Using Item Ranks During Decoding

ISD uses the ranked item set $Q(h)$ before filtering and after SID completion. Both steps combine the decoder rank with the item rank using reciprocal rank fusion [1].

5.2.1 Before Filtering. Section 4 tests every item beneath each possible next token. ISD approximates this test with $Q(h)$. At each semantic position, let $r_{\text{dec}}(p)$ be the rank of extended SID prefix p under the decoder’s cumulative sequence probability. Using S_p for the items whose SIDs begin with p , we give p the rank of its highest-ranked item in $Q(h)$.

$$r_{\text{item}}(p) = \min_{i \in Q(h) \cap S_p} r_Q(i), \quad (15)$$

where $r_Q(i)$ is the rank of i in $Q(h)$. If $Q(h) \cap S_p$ is empty, we set $r_{\text{item}}(p) = \infty$. We then combine the decoder rank and item rank using reciprocal rank fusion [1]:

$$F(p) = \frac{1}{\kappa + r_{\text{dec}}(p)} + \frac{1[r_{\text{item}}(p) < \infty]}{\kappa + r_{\text{item}}(p)}. \quad (16)$$

Here, κ is the smoothing constant. Reciprocal-rank fusion places the two rankings on a common scale without learning a calibration between their raw scores [1]. We retain the B SID prefixes with the largest $F(p)$ and apply the same rule at every semantic position. The first term favors a prefix ranked highly by the decoder. The second favors a prefix containing a highly ranked item from $Q(h)$. Appendix A.3 reports J , B , κ , and the history windows.

Table 3: Recommendation results. Each % Improvement row compares the ISD variant immediately above it with the corresponding generative recommender. Bold and underlined values indicate the best and second-best results in each column. N@K and R@K denote NDCG and Recall at cutoff K .

Method	Scientific				Office				Baby			
	N@10	N@20	R@10	R@20	N@10	N@20	R@10	R@20	N@10	N@20	R@10	R@20
SASRec [8]	0.0844	0.0926	0.1183	0.1509	0.1008	0.1062	0.1282	0.1495	0.0347	0.0420	0.0623	0.0912
BERT4Rec [19]	0.0679	0.0745	0.0952	0.1214	0.0909	0.0951	0.1143	0.1313	0.0185	0.0240	0.0359	0.0580
UniSRec [6]	0.0842	0.0910	0.1169	0.1440	0.1046	0.1102	0.1342	0.1564	0.0296	0.0376	0.0653	0.0972
SETRec [11]	0.0822	0.0904	0.1190	0.1518	0.1032	0.1086	0.1313	0.1530	0.0353	0.0426	0.0652	0.0942
ETEGRec [12]	0.0783	0.0860	0.1108	0.1414	0.1024	0.1075	0.1298	0.1486	0.0337	0.0412	0.0637	0.0933
Statistics ranker [16, 18]	0.0824	0.0882	0.1098	0.1331	0.1016	0.1049	0.1247	0.1381	0.0299	0.0337	0.0479	0.0633
TIGER [15]	0.0802	0.0887	0.1167	0.1506	0.0949	0.1005	0.1194	0.1415	0.0295	0.0371	0.0565	0.0866
+ ISD (statistics)	0.0891	0.0947	0.1235	0.1460	<u>0.1074</u>	<u>0.1119</u>	0.1349	0.1527	0.0349	0.0391	0.0579	0.0745
% Improvement	+11.0%	+6.8%	+5.8%	-3.1%	+13.1%	+11.3%	+13.0%	+7.9%	+18.2%	+5.5%	+2.5%	-14.0%
+ ISD (SASRec)	0.0875	0.0957	0.1277	0.1604	0.1026	0.1081	0.1332	0.1549	0.0365	0.0444	0.0686	0.0997
% Improvement	+9.1%	+7.9%	+9.5%	+6.5%	+8.1%	+7.5%	+11.5%	+9.5%	+23.9%	+19.7%	+21.7%	+15.1%
+ ISD (UniSRec)	0.0885	0.0965	0.1257	0.1572	<u>0.1048</u>	<u>0.1109</u>	0.1367	0.1606	0.0387	0.0482	0.0716	<u>0.1093</u>
% Improvement	+10.3%	+8.7%	+7.8%	+4.4%	+10.4%	+10.3%	+14.5%	+13.5%	+31.2%	+30.0%	+27.1%	+26.2%
LIGER [23]	0.0880	0.0970	0.1291	0.1650	0.0983	0.1038	0.1257	0.1476	0.0398	0.0480	0.0726	0.1053
+ ISD (statistics)	0.0927	0.0986	0.1266	0.1499	0.1081	0.1126	<u>0.1357</u>	0.1534	0.0403	0.0483	0.0734	0.1052
% Improvement	+5.3%	+1.6%	-1.9%	-9.1%	+10.1%	+8.5%	+8.0%	+3.9%	+1.1%	+0.5%	+1.2%	-0.1%
+ ISD (SASRec)	0.0914	<u>0.1010</u>	<u>0.1335</u>	<u>0.1714</u>	0.1010	0.1075	0.1314	0.1571	<u>0.0406</u>	<u>0.0491</u>	<u>0.0745</u>	0.1082
% Improvement	+3.8%	+4.1%	+3.4%	+3.9%	+2.8%	+3.6%	+4.6%	+6.4%	+1.8%	+2.0%	+2.5%	+2.6%
+ ISD (UniSRec)	<u>0.0921</u>	0.1013	0.1356	0.1724	0.1011	0.1081	0.1317	<u>0.1593</u>	0.0416	0.0509	0.0769	0.1137
% Improvement	+4.6%	+4.4%	+5.1%	+4.5%	+2.9%	+4.1%	+4.8%	+7.9%	+4.5%	+5.8%	+5.9%	+7.8%

5.2.2 After SID Completion. Preserving an SID prefix keeps its items available but does not place them near the top of the final recommendation list. Let $r_{\text{SID}}(i)$ be generated item i 's rank under the decoder and set $r_Q(i) = \infty$ when $i \notin Q(h)$. We rank the generated items by

$$F_{\text{item}}(i) = \frac{1}{\kappa + r_{\text{SID}}(i)} + \frac{1[r_Q(i) < \infty]}{\kappa + r_Q(i)}. \quad (17)$$

ISD constructs $Q(h)$ once and uses one decoder pass. It does not change an SID or insert an item whose SID was not generated.

5.3 Complexity Analysis

We treat the ranked item set $Q(h)$ as a preprocessing input to ISD. Its construction is ranker-dependent and is therefore excluded from the decoding complexity. Let T_{dec} denote the cumulative cost of the decoder forward passes. Ordinary SID generation considers at most BV prefix extensions at each of L positions, giving runtime

$$O(T_{\text{dec}} + LBV). \quad (18)$$

Given $Q(h)$, ISD maps its J items to their L prefixes, checks item support for at most LBV extensions, and orders the B generated items. Its decoding runtime is

$$O(T_{\text{dec}} + LBV + JL + B \log B). \quad (19)$$

For a fixed decoding configuration, the item-support budget J , SID length L , beam width B , and maximum number of next-token choices V are constants. ISD therefore has the same asymptotic decoding complexity as ordinary SID generation.

6 EXPERIMENTS

In this section, we evaluate Item-Supported Decoding (ISD) on three domains from the Amazon Reviews dataset [13]. We use

the following research questions to guide our experiments: **RQ1:** Does ISD improve recommendation performance? **RQ2:** How much of the improvement comes from supporting SID prefixes during generation, and how much comes from ordering the generated items afterward? **RQ3:** When the same item is assigned an alternative SID after an equivalent change to its description, can ISD keep that alternative SID available during generation?

6.1 Datasets and Experimental Details

6.1.1 Datasets and Splits. We use the Baby, Office, and Scientific domains from the Amazon Reviews dataset [13]. We order each user's interactions chronologically, reserve the final interaction for testing and the preceding interaction for validation, and use the remainder for training. This follows the leave-one-out protocol used by the evaluated sequential and generative recommenders [8, 15]. Appendix Table 6 reports the processed dataset statistics.

6.1.2 Compared Methods. We include three sequential item recommenders: SASRec [8], BERT4Rec [19], and UniSRec [6]. SETRec [11] avoids left-to-right SID filtering through order-agnostic generation, while ETEGRec [12] jointly trains its item tokenizer and generative recommender. TIGER [15] and LIGER [23] are the autoregressive SID systems to which we apply ISD. For the item ranking $Q(h)$, we compare the training-statistics score in Eq. equation 14, which combines first-order transitions and item-based co-consumption [16, 18], with SASRec [8] and UniSRec [6]. The corresponding rows are denoted ISD (statistics), ISD (SASRec), and ISD (UniSRec).

6.1.3 Evaluation Protocol. For each held-out interaction, we rank the target item against all other eligible items in the domain and report Recall and NDCG at cutoffs 10 and 20. All methods use

Table 4: Held-out items remaining after each token under alternative SID assignments (%).

Method	Office			Scientific		
	T1	T2	T3	T1	T2	T3
TIGER [15]	18.1	0.04	0.00	18.0	0.05	0.00
+ ISD (statistics)	31.2	13.8	13.5	33.2	14.4	13.5
+ ISD (SASRec)	33.3	14.6	13.8	35.9	15.2	13.8
+ ISD (UniSRec)	34.0	15.3	14.2	33.8	14.3	13.1
LIGER [23]	17.8	0.04	0.00	16.0	0.10	0.00
+ ISD (statistics)	31.1	13.7	13.5	32.5	14.4	13.6
+ ISD (SASRec)	33.2	14.6	13.8	35.4	15.1	13.7
+ ISD (UniSRec)	33.8	15.3	14.2	33.5	14.6	13.2

the same interaction histories and eligible items. Appendix A.4 describes the baseline adaptations, model selection, and three-seed evaluation. Appendix A.3 reports the fixed ISD settings.

6.2 Recommendation Results (RQ1)

Table 3 reports the matched comparison and varies the source of the item ranking. Across the three-domain grid, all three sources numerically improve NDCG@10 for both evaluated SID systems. The effect is largest for TIGER on Baby, where training statistics, SASRec, and UniSRec improve NDCG@10 by 18.2%, 23.9%, and 31.2%, respectively. LIGER’s Baby gains are smaller, ranging from 1.1% to 4.5%. The improvement therefore does not depend on the hand-designed training statistics: rankings supplied by SASRec and UniSRec also improve SID generation without retraining its decoder.

The provider matters for the remaining metrics. With TIGER, training statistics improve NDCG@10 but reduce Recall@20 on Scientific and Baby, whereas SASRec and UniSRec improve all four metrics on all three domains. With LIGER on Baby, the statistics and SASRec gains are modest, while UniSRec gives the largest improvement. These differences are consistent with ISD’s role: it preserves and orders the items favored by its provider rather than defining a new source of relevance.

6.3 Ablation Study (RQ2)

ISD uses the item ranking twice: first to support SID prefixes before beam filtering and then to order the items whose complete SIDs were generated. We separate these uses for both SID systems on their matched validation populations in Table 5.

6.4 Generation Under SID Reassignment (RQ3)

Table 4 shows that TIGER and LIGER preserve almost none of the reassigned SID paths through the third semantic token, whereas ISD preserves 13.1–14.2% across item-ranking methods and domains. Item support therefore helps an existing decoder follow a valid semantic path that differs from the target’s original label. Complete retrieval remains zero because the frozen decoder has not learned the reassigned collision-resolution suffix, so ISD improves robustness to semantic reassignment but does not solve exact identity remapping.

Table 5: Ablation of the two ISD stages. “Filtering” acts before beam reduction, while “ordering” acts after SID generation.

Backbone	ISD stage	Office		Scientific	
		N@10	R@10	N@10	R@10
TIGER	No ISD	0.0949	0.1194	0.0802	0.1166
	Filtering only	0.0950	0.1194	0.0802	0.1167
	Ordering only	0.1035	0.1294	0.0887	0.1250
	Filtering + ordering	0.1074	0.1349	0.0891	0.1235
LIGER	No ISD	0.0983	0.1257	0.0880	0.1291
	Filtering only	0.1015	0.1323	0.0934	0.1374
	Ordering only	0.1026	0.1292	0.0910	0.1289
	Filtering + ordering	0.1081	0.1357	0.0927	0.1266

The useful stage depends on the downstream head. TIGER receives most of its gain from ordering, and combining filtering with ordering is strongest on Office. LIGER already has a dense item head: filtering alone is strongest on Scientific, while combining both stages gives the highest Office NDCG@10. Thus filtering and ordering are distinct interventions rather than interchangeable implementations of the same score.

6.5 Interpretation and Scope

The completed comparison supports a limited but useful conclusion. Item evidence can improve an already-trained autoregressive SID system at inference time, and the effect is not specific to one hand-designed ranking rule. ISD does not replace the SID decoder. It changes which SID prefixes remain under the fixed beam and how the generated items are ordered. It also does not address unstable exact item identity from Section 3.

The comparison averages three runs per trainable method after validation-only checkpoint selection. We therefore use it to test the design consequence of Sections 3–4, not to claim that ISD universally improves every SID system or replaces conventional item recommendation.

7 RELATED WORK

SID-based generative recommendation. A generative recommender predicts an item by generating its identifier rather than scoring every item independently. TIGER generates short residual-quantized identifiers from user histories [15]. Later methods incorporate collaborative signals, align tokenization with recommendation, or scale SID-based models [2, 4, 9, 12, 21]. These works improve SID learning or generation. We instead examine how SID construction changes item representations and how its tokens determine which items remain available.

SID decoding and item selection. Decoding-focused methods address premature removal or sequential token dependence. APAO retrains the decoder with prefix-level objectives [24]. RPG and SETRec generate order-agnostic tokens in parallel [5, 11], while LIGER combines SID generation with dense retrieval and ranking [23]. ISD instead keeps the SID assignment and decoder fixed, using an independent item ranking to support prefixes before beam reduction and to order the generated items.

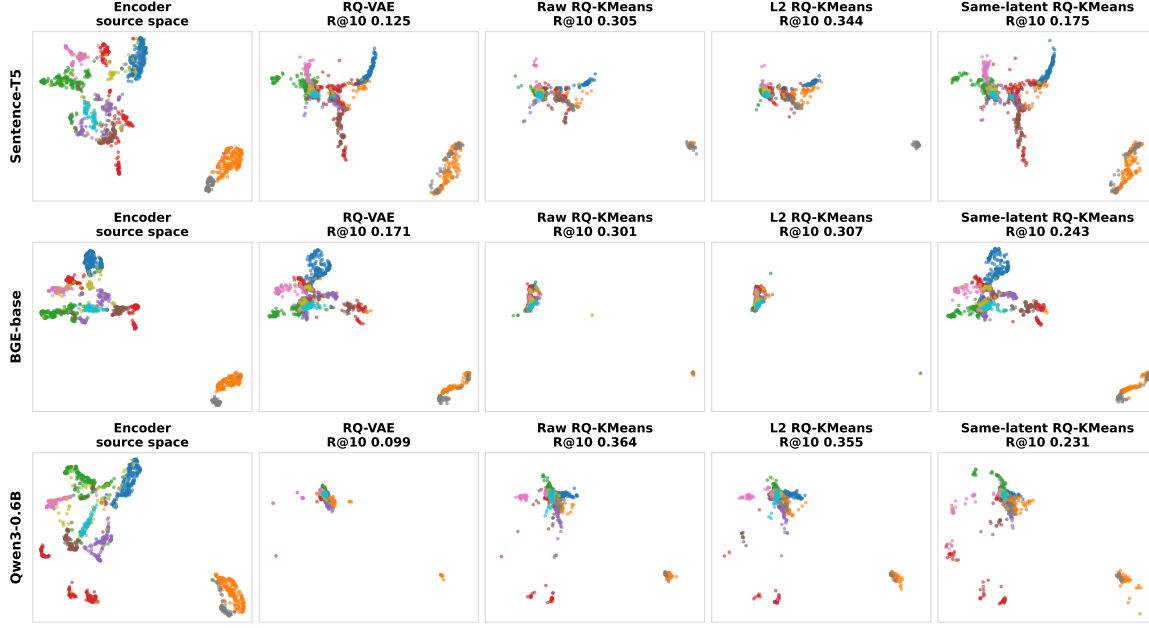
Quantization and representation evaluation. SID construction builds on product and learned vector quantization [3, 7, 10, 20]. Conventional retrieval uses a quantized code to locate candidates and a separate key to identify the item. SID recommendation places both roles in one generated sequence. Our controlled tests therefore distinguish continuous item recognition, neighborhood preservation, and exact SID consistency.

8 CONCLUSION

SIDs provide useful coarse organization, but their exact codes are not determined by semantics alone, and their fine tokens preserve limited local item structure while deciding which items survive generation. ISD reduces this conflict by adding item-level evidence before beam reduction and after generation, improving NDCG@10 by up to 31.2% across TIGER and LIGER without retraining. SIDs can organize and identify items, but their fine boundaries should not alone control item selection.

REFERENCES

- [1] Gordon V. Cormack, Charles L A Clarke, and Stefan Buettcher. 2009. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '09)*. Association for Computing Machinery, New York, NY, USA, 758–759. <https://doi.org/10.1145/1571941.1572114>
- [2] Junchen Fu, Xuri Ge, Alexandros Karatzoglou, Ioannis Arapakis, Suzan Verberne, Joemon M. Jose, and Zhaochun Ren. 2026. Differentiable Semantic ID for Generative Recommendation. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '26)*. Association for Computing Machinery, New York, NY, USA, 369–379. <https://doi.org/10.1145/3805712.3809641>
- [3] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2013. Optimized Product Quantization for Approximate Nearest Neighbor Search. In *Proceedings of the 2013 IEEE Conference on Computer Vision and Pattern Recognition (CVPR '13)*. IEEE Computer Society, USA, 2946–2953. <https://doi.org/10.1109/CVPR.2013.379>
- [4] Yingzhi He, Yan Sun, Junfei Tan, Yuxin Chen, Xiaoyu Kong, Chunxu Shen, Xiang Wang, An Zhang, and Tat-Seng Chua. 2026. Reasoning over Semantic IDs Enhances Generative Recommendation. [arXiv:cs.IR/2603.23183](https://arxiv.org/abs/2603.23183) <https://arxiv.org/abs/2603.23183>
- [5] Yupeng Hou, Jiacheng Li, Ashley Shin, Jinsung Jeon, Abhishek Santhanam, Wei Shao, Kaveh Hassani, Ning Yao, and Julian McAuley. 2025. Generating Long Semantic IDs in Parallel for Recommendation. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '25)*. Association for Computing Machinery, New York, NY, USA, 956–966. <https://doi.org/10.1145/3711896.3736979>
- [6] Yupeng Hou, Shanlei Mu, Wayne Xin Zhao, Yaliang Li, Bolin Ding, and Ji-Rong Wen. 2022. Towards Universal Sequence Representation Learning for Recommender Systems. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '22)*. Association for Computing Machinery, New York, NY, USA, 585–593. <https://doi.org/10.1145/3534678.3539381>
- [7] Herve Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. <https://doi.org/10.1109/TPAMI.2010.57>
- [8] Wang-Cheng Kang and Julian McAuley. 2018. Self-Attentive Sequential Recommendation. [arXiv:cs.IR/1808.09781](https://arxiv.org/abs/1808.09781) <https://arxiv.org/abs/1808.09781>
- [9] Xiaoyu Kong, Leheng Sheng, Junfei Tan, Yuxin Chen, Jiancan Wu, An Zhang, Xiang Wang, and Xiangnan He. 2025. MiniOneRec: An Open-Source Framework for Scaling Generative Recommendation. [arXiv:cs.IR/2510.24431](https://arxiv.org/abs/2510.24431) <https://arxiv.org/abs/2510.24431>
- [10] Doyup Lee, Chiheon Kim, Saehoon Kim, Minsu Cho, and Wook-Shin Han. 2022. Autoregressive Image Generation using Residual Quantization. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 11513–11522. <https://doi.org/10.1109/CVPR52688.2022.01123>
- [11] Xinyu Lin, Haihan Shi, Wenjie Wang, Fuli Feng, Qifan Wang, See-Kiong Ng, and Tat-Seng Chua. 2025. Order-agnostic Identifier for Large Language Model-based Generative Recommendation. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '25)*. Association for Computing Machinery, New York, NY, USA, 1923–1933. <https://doi.org/10.1145/3726302.3730053>
- [12] Enze Liu, Bowen Zheng, Cheng Ling, Lantao Hu, Han Li, and Wayne Xin Zhao. 2025. Generative Recommender with End-to-End Learnable Item Tokenization. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '25)*. Association for Computing Machinery, New York, NY, USA, 729–739. <https://doi.org/10.1145/3726302.3729989>
- [13] Jianmo Ni, Jiacheng Li, and Julian McAuley. 2019. Justifying Recommendations using Distantly-Labeled Reviews and Fine-Grained Aspects. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan (Eds.). Association for Computational Linguistics, Hong Kong, China, 188–197. <https://doi.org/10.18653/v1/D19-1018>
- [14] Gustavo Penha, Edoardo D'Amico, Marco De Nadai, Enrico Palumbo, Alexandre Tamborrino, Ali Vardasbi, Max Lefarov, Shawn Lin, Timothy Heath, Francesco Fabbri, and Hugues Bouchard. 2025. Semantic IDs for Joint Generative Search and Recommendation. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems (RecSys '25)*. Association for Computing Machinery, New York, NY, USA, 1296–1301. <https://doi.org/10.1145/3705328.3759300>
- [15] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q. Tran, Jonah Samost, Maciej Kula, Ed H. Chi, and Maheswaran Sathiamoorthy. 2023. Recommender Systems with Generative Retrieval. In *Thirty-seventh Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=BJ0fQUU32w>
- [16] Steffen Rendle, Christoph Freudenthaler, and Lars Schmidt-Thieme. 2010. Factorizing personalized Markov chains for next-basket recommendation. In *Proceedings of the 19th International Conference on World Wide Web (WWW '10)*. Association for Computing Machinery, New York, NY, USA, 811–820. <https://doi.org/10.1145/1772690.1772773>
- [17] Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. 2020. Beyond Accuracy: Behavioral Testing of NLP Models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault (Eds.). Association for Computational Linguistics, Online, 4902–4912. <https://doi.org/10.18653/v1/2020.acl-main.442>
- [18] Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. 2001. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th International Conference on World Wide Web (WWW '01)*. Association for Computing Machinery, New York, NY, USA, 285–295. <https://doi.org/10.1145/371920.372071>
- [19] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM '19)*. Association for Computing Machinery, New York, NY, USA, 1441–1450. <https://doi.org/10.1145/3357384.3357895>
- [20] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. 2017. Neural discrete representation learning. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS'17)*. Curran Associates Inc., Red Hook, NY, USA, 6309–6318.
- [21] Wenjie Wang, Honghui Bao, Xinyu Lin, Jizhi Zhang, Yongqi Li, Fuli Feng, See-Kiong Ng, and Tat-Seng Chua. 2024. Learnable Item Tokenization for Generative Recommendation. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM '24)*. Association for Computing Machinery, New York, NY, USA, 2400–2409. <https://doi.org/10.1145/3627673.3679569>
- [22] Xuesi Wang, Junjie Wang, Ziliang Wang, Weijie Bian, and Guanxing Zhang. 2026. Decoupled Residual Quantization for Robust Semantic IDs in Recommendation. [arXiv:cs.IR/2606.01844](https://arxiv.org/abs/2606.01844) <https://arxiv.org/abs/2606.01844>
- [23] Liu Yang, Fabian Paischer, Kaveh Hassani, Jiacheng Li, Shuai Shao, Zhang Gabriel Li, Yun He, Xue Feng, Nima Noorshams, Sem Park, Bo Long, Robert D Nowak, Xiaoli Gao, and Hamid Eghbalzadeh. 2025. Unifying Generative and Dense Retrieval for Sequential Recommendation. *Transactions on Machine Learning Research* (2025). <https://openreview.net/forum?id=xjdnFlsjCb>
- [24] Yuanqing Yu, Yifan Wang, Weizhi Ma, Zhiqiang Guo, and Min Zhang. 2026. APAO: Adaptive Prefix-Aware Optimization for Generative Recommendation. [arXiv:cs.IR/2603.02730](https://arxiv.org/abs/2603.02730) <https://arxiv.org/abs/2603.02730>



Office: rigidly aligned encoder rows; every panel contains the same 1,200 items, with overlaps overplotted.

Figure 5: Encoder space and SID-reconstructed geometry on Office. Broad category organization remains visible, while fine item neighborhoods change.

A EXPERIMENTAL AND IMPLEMENTATION DETAILS

A.1 Section 3: Representation Measurements

§ 3 compares matched descriptions across Sentence-T5, BGE, and Qwen encoders. The controlled constructor comparison holds the Sentence-T5 representation fixed so that only SID construction changes. Table 2 lists the evaluated constructors. Complete-system checks additionally cover TIGER, RPG, DIGER, LIGER, SIDReasoner, and MiniOneRec.

A.1.1 Matched-Description Protocol. For every item, the source and reordered descriptions contain the same words and attribute values. Character-level and token-multiset checks verify that the transformation adds, removes, or changes no item information. An item enters a constructor comparison only when the implementation exactly reproduces its stored source SID. This prevents reproduction errors from being counted as SID changes.

The primary comparison covers Baby, Office, and Scientific. It uses one pinned Sentence-T5 export per domain and ten frozen constructor seeds. Reported means average over seeds. Confidence intervals resample both items and seeds. All nearest-neighbor searches exclude the query item. Encoder neighborhoods use cosine similarity, while SID neighborhoods use Euclidean distance between vectors reconstructed from the codebooks.

A.1.2 Additional Local-Structure Checks. Exact neighbor overlap is intentionally strict. We therefore also measure rank-based trustworthiness and continuity, encoder-cosine regret, and category agreement. Across the evaluated constructors, trustworthiness and continuity range from 0.950 to 0.994 and cosine regret ranges from 0.043 to 0.104. Thus, items replacing a missing exact neighbor are often still nearby in the encoder space. These checks support the main-text conclusion that SIDs retain broad organization while changing many fine relationships. They do not imply that the reconstructed neighborhoods are arbitrary.

A.1.3 Geometry Visualizations. Figures 5 and 6 shows the encoder space and the corresponding SID reconstructions for Sentence-T5, BGE, and Qwen. Within each row, the projection, items, and category colors are fixed across constructors. The panels should therefore be compared within a row. They provide a qualitative view of the quantitative neighborhood results: broad category organization remains visible, while many local item relationships change.

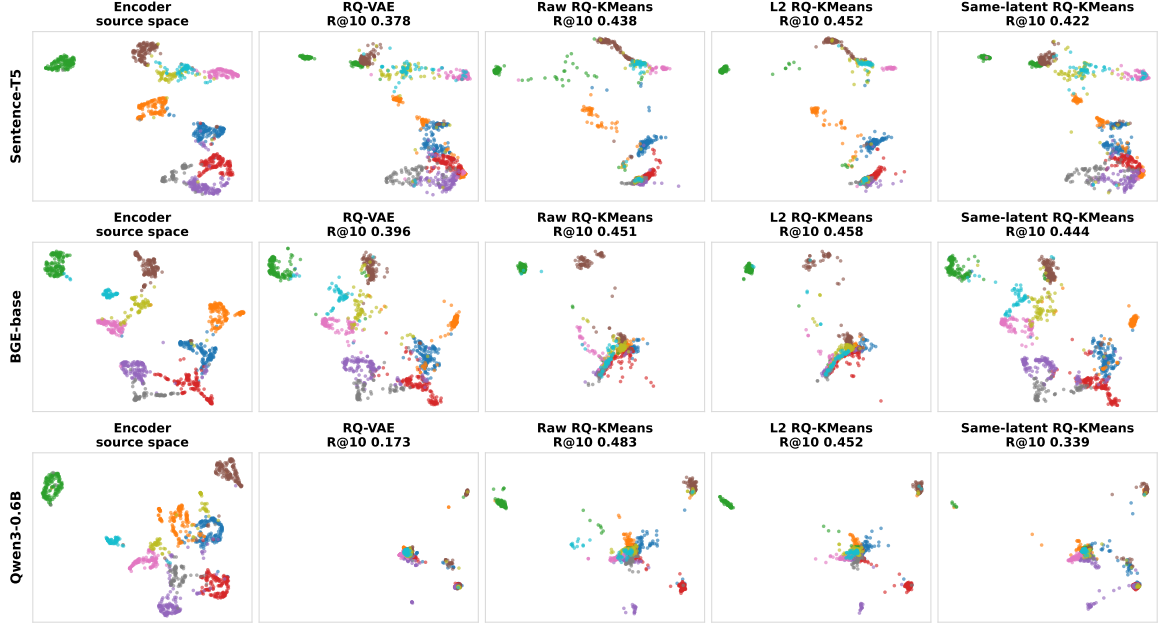
A.1.4 Organization Across SID Positions. Before token z_ℓ is assigned, let $p = (z_1, \dots, z_{\ell-1})$ denote the existing SID prefix. Let S_p contain all items whose SIDs begin with p , and let $S_{p,v} \subseteq S_p$ contain those assigned next token v . We measure the separation introduced at position ℓ by

$$I_{p,\ell} = \sum_v \frac{|S_{p,v}|}{|S_p|} \log_2 \frac{|S_p|}{|S_{p,v}|}. \quad (20)$$

This is the Shannon entropy of the empirical next-token distribution. It is also the expected reduction in the log-size of the remaining item set.

To measure whether the split keeps encoder neighbors together, let $N_E^{10}(i)$ be item i 's self-excluded top-10 encoder neighbors. For every non-singleton child group, we compute the fraction of these neighbors that receive the same next token as i . We compare the observed fraction with 200 randomized assignments within the same parent prefix. Each randomization preserves the parent membership, the number and size of child groups, and their item-popularity composition. The excess over this randomized baseline is the source-neighbor organization introduced by the observed split.

We aggregate separation and excess organization across prefixes using the number of items under each prefix as weights. Their ratio gives the source-neighbor organization per bit reported in Figure 3. Parent-balanced and common-support variants give the same qualitative pattern. Uncertainty is estimated by bootstrapping parent prefixes within constructor seeds. The collision-resolution suffix is excluded throughout.



Scientific: rigidly aligned encoder rows; every panel contains the same 1,200 items, with overlaps overplotted.

Figure 6: Encoder space and SID-reconstructed geometry on Scientific. Broad category organization remains visible, while fine item neighborhoods change.

A.2 Section 4: Target Filtering

§ 4 follows each held-out target through SID generation. The primary diagnostic uses TIGER-style decoders with RQ-VAE SIDs, beam width 50, and decoder seeds 42–44. The figure also reports the corresponding validation-selected LIGER result. At each semantic position, a target remains available if at least one retained SID prefix agrees with the target’s assigned prefix through that position.

A.2.1 Top- k Target Population. A separate sequential item scorer is trained only on training interactions. Before SID generation, it ranks every eligible unconsumed item from the observed history. We retain histories for which this scorer places the held-out target among its top k items. This isolates targets that could plausibly appear near the top of a recommendation list before SID filtering. The reported experiment uses $k = 10$. Its fixed population contains 1,198 Office histories with 345 distinct targets and 536 Scientific histories with 214 distinct targets. All position-wise comparisons use these same histories.

The unweighted metric K_ℓ in § 4.2 counts the fraction of this population still available after semantic position ℓ . For a rank-sensitive check, let $r_h \leq k$ be the target’s rank under the unfiltered item scorer and set $w_h = 1/\log_2(1 + r_h)$. We report

$$V_\ell = \frac{\sum_{h \in \mathcal{H}_k} w_h \mathbf{1}[i^*(h) \in \mathcal{R}_\ell]}{\sum_{h \in \mathcal{H}_k} w_h}. \quad (21)$$

This gives more weight to the removal of targets that the independent scorer would place closer to first.

A.2.2 Full-Information Intervention. The diagnostic intervention scores every eligible item beneath every possible next token before beam reduction.

For a candidate extension (p, v) , its item score is the largest score among unconsumed items whose SIDs begin with (p, v) . We standardize this value and the decoder score across the possible next tokens, combine them with a weight selected on validation data, and then apply the original beam budget. A zero-variance signal is set to zero. Validation selects weight $\alpha = 0.75$.

The held-out target is never identified, inserted, or restored. However, the intervention requires scoring every eligible item before every token decision. It is therefore used only to test whether item-level evidence can prevent removal. Section 5 evaluates the efficient approximation.

A.3 Section 5: Item-Supported Decoding

A.3.1 ISD Settings. The ranked item set $Q(h)$ contains $J = 50$ items and is constructed once per history. The SID decoder retains $B = 50$ prefixes at each of three semantic positions and returns at most 50 items. Reciprocal-rank fusion uses $\kappa = 60$. These settings are fixed across datasets, SID systems, and item ranking providers.

Training-statistics scores combine last-item transitions with co-consumption over the ten most recent items. SASRec and UniSRec also provide $Q(h)$.

A.3.2 Training-Statistics Normalization. Let E_h be the eligible unconsumed items for history h , and let u_i be one of the two training-statistics signals for item i . We compute

$$\mu_h(u) = \frac{1}{|E_h|} \sum_{j \in E_h} u_j, \quad \sigma_h(u) = \sqrt{\frac{1}{|E_h|} \sum_{j \in E_h} (u_j - \mu_h(u))^2}. \quad (22)$$

The standardized value used in Eq. equation 14 is

$$z_h(u_i) = \begin{cases} (u_i - \mu_h(u))/\sigma_h(u), & \sigma_h(u) > 0, \\ 0, & \sigma_h(u) = 0. \end{cases} \quad (23)$$

A.4 Section 6: Recommendation Evaluation

Table 6: Dataset statistics after preprocessing.

Dataset	Items	Users	Training interactions
Scientific	5,334	11,024	54,958
Office	27,965	101,478	597,310
Baby	7,050	19,445	121,902

A.4.1 Data and Model Selection. Interactions are ordered chronologically. The last interaction is held out for testing, the preceding interaction

is used for validation, and the remainder forms the training set. All compared methods use the same histories, eligible items, and consumed-item mask. Model checkpoints and ISD providers are selected using validation NDCG@10.

A.4.2 Evaluation Protocol. We retain each baseline’s model architecture, training objective, and recommended hyperparameters while using the common data splits and item masks. We train each method with three random seeds, select checkpoints on validation data, and report the arithmetic mean across seeds. The test set is evaluated only after model selection. For each held-out interaction, we rank the target against all eligible items and report Recall and NDCG at cutoffs 10 and 20. Every method returns at most 50 items.